Ensemble Methods: Boosting vs. Bagging

AI Academy — Machine Learning Final Project, Spring 2026

Emin Huseynli · Ziyad Muradov · Ismayil Yusifli

github.com/thedarkstringg/ml-final • tag: v1.0-final

Outline

- 1. Motivation & central question
- 2. Module 1 — Decision Tree (CART)
- 3. Module 2 — AdaBoost (SAMME)
- 4. Module 3 — Random Forest
- 5. Module 4 — Unsupervised pipeline (PCA, K-Means, DBSCAN)
- 6. Datasets & experimental design
- 7. Results: 7 experiments
- 8. Key findings & conclusion

Motivation & Central Question

*"Under what conditions does boosting outperform bagging, and vice versa — and **why**?"*

Boosting (AdaBoost / SAMME)	Bagging (Random Forest)
Re-weights hard examples	Bootstrap aggregation
Reduces bias sequentially	Reduces variance in parallel
Fast convergence on clean data	Robust to label noise
Sensitive to class imbalance	Handles imbalance better

Module 1 — Decision Tree (CART)

- Binary CART classifier implemented from scratch (NumPy only)
- Supports Gini impurity and information gain criteria
- Weighted samples — required by AdaBoost (SAMME)
- max_features subsampling — required by Random Forest
- Feature importances via normalized impurity reduction
- Verified ≤ 2% difference from sklearn on all datasets

Gini: $I_G(p) = 1 - \Sigma\, p_c^2$ Split gain: $\Delta I = I(\text{parent}) - (N_L/N)\cdot I_L - (N_R/N)\cdot I_R$

Module 2 — AdaBoost (SAMME)

- Discrete SAMME variant — supports multiclass natively
- Weak learner: depth-1 DecisionTree (decision stump)
- Weighted impurity: $p_c = \Sigma_{i:y_i=c} w_i / \Sigma_i w_i$
- Estimator weight: $\alpha_m = \ln((1-\varepsilon_m)/\varepsilon_m) + \ln(K-1)$
- Weight update: $w_i \propto w_i \cdot \exp(\alpha_m \cdot 1[h_m(x_i) \neq y_i])$
- staged_predict, estimator_weights, estimator_errors implemented
- Bonus: SAMME.R (real-valued variant) also implemented

Module 3 — Random Forest

- Bootstrap aggregation over T independent decision trees
- Feature subsampling: ■√p■ features per split by default
- Majority vote (predict) / averaged probabilities (predict_proba)
- Out-of-bag (OOB) score for free generalization estimate
- Parallelism: multiprocessing.Pool when n_jobs > 1
- Feature importances: averaged across all trees

Module 4 — Unsupervised Pipeline

- PCA: eigen-decomposition of covariance matrix; scree plot
- K-Means: Lloyd's algorithm with k-means++ initialization
- → Elbow method (k = 1…10, 10 restarts, best inertia)
- DBSCAN: density-based clustering; ε chosen from k-distance knee
- All three: ARI reported against true labels (via sklearn.metrics)
- 2D PCA scatter colored by: true class / K-Means / DBSCAN labels

Datasets & Experimental Design

Dataset	Task	Samples	Features	Highlight
Breast Cancer Wisconsin	Binary	569	30	Baseline benchmark
Adult Income	Binary	48,842	14	Class imbalance → SMOTE
Covertype (subset)	Multi-class	≥ 5,000	54	OvR AdaBoost
MNIST (2-class subset)	Binary	≥ 5,000	784	High-dimensional

- All seeds fixed at 42; reproducible via: python src/experiments/run_all.py

Results — Experiments 1–5

- Exp 1 (Baseline): our DecisionTree matches sklearn within ≤ 2% on all datasets
- Exp 2 (AdaBoost scaling): test error plateaus then slightly rises — see figures/exp2_*
- Exp 3 (RF scaling): OOB accuracy tracks test accuracy closely
- Exp 4 (Head-to-head, 5-fold CV): RF dominates on imbalanced / noisy sets
- Exp 5 (Noise robustness): AdaBoost accuracy drops faster under η = 10–20% label noise

[Figures from figures/exp1_* through figures/exp5_* inserted in report]

Results — Experiments 6–7

- Exp 6 (Bias-variance, B=100 bootstrap replicates):
- → AdaBoost: lower bias², higher variance
- → Random Forest: higher bias², lower variance
- Exp 7 (Unsupervised analysis):
- → PCA scree: ≥ 90% variance in first k components
- → K-Means ARI aligns well with class boundaries on clean data
- → DBSCAN captures non-convex structure; K-Means misses it

[Figures: scree plot, PCA scatter, k-distance plot in figures/exp6_* exp7_*]

Bonus Contributions

- SAMME.R (+2 pts): real-valued variant, improved probability calibration
- → src/boosting/samme_r.py | experiments/bonus_samme_r.py
- t-SNE visualization (+2 pts): comparison with PCA projections
- → experiments/bonus_tsne.py
- Both bonuses are tested, documented, and integrated into run_all.py

Key Findings & Conclusion

- Boosting wins: clean, balanced, binary datasets — lower bias
- Bagging wins: noisy labels, class imbalance, high-dim — lower variance
- Unsupervised: PCA + K-Means clusters align with classes on clean data;
- DBSCAN finds non-convex clusters that K-Means misses
- All 7 experiments fully reproducible; code matches sklearn within 2%

Thank you — Questions?